# Lecture Notes

Monotone Classification with Relative Approximations

Your Name
Your University

March 21, 2026

**Abstract**

Brief summary of what these notes cover. Replace this with your own abstract or remove this block entirely.

## Contents

**1 Introduction** **2**

**2 Problem Definitions** **2**
2.1 Math Conventions . . . . . 2
2.2 Dominance . . . . . 3

**3 Results** **3**

**4 Algorithms** **3**

**5 Proofs** **3**

# 1 Introduction

Write your introductory text here. Below is a quick demo of every reusable box. Copy–paste whichever you need.

**Definition 1.1** (Monotone Classifier)**.** A classifier $h : \mathbb{R}^d \to \{-1, 1\}$ is *monotone* if $h(p) \geq h(q)$ for every $p, q \in \mathbb{R}^d$ satisfying $p \succ q$.

**Theorem 1.1** (Upper Bound)**.** For any $\epsilon > 0$, there exists an algorithm that finds a $(1 + \epsilon)$-approximate monotone classifier using $O\left(\frac{\log n}{\epsilon}\right)$ label queries.

*Proof.* Write your proof here. The □ is inserted automatically. □

**Lemma 1.1.** For any input $P$ and monotone classifier $h \in \mathbb{H}_{\text{mon}}$,

$$\text{err}_P(h) \ \geq \ k^*.$$

*Proof.* Proof of the lemma. □

**Proposition 1.1.** If $P$ is monotone then $k^* = 0$.

**Corollary 1.1.** Theorem 1.1 implies that the algorithm of ... achieves an error of at most $(1 + \epsilon)k^*$ with high probability.

**Problem 1.1** (Monotone Classification with Relative Precision)**.** Given $\epsilon \geq 0$, find a monotone classifier whose error on $P$ is at most $(1 + \epsilon)k^*$. The efficiency of an algorithm is measured by the number of elements probed.

**Example 1.1.** Let $P = \{p_1, p_2, \ldots, p_n\} \subset \mathbb{R}^2$. The optimal monotone classifier $h$ misclassifies only $p_1$, so $k^* = 1$.

**Remark 1.1.** When $\epsilon = 0$ the problem reduces to finding an *optimal* monotone classifier, which is achievable in polynomial time after revealing all labels.

# 2 Problem Definitions

## 2.1 Math Conventions

For an integer $x \geq 1$, the notation $[x]$ denotes $\{1, 2, \ldots, x\}$. We write $x \leq_{1+\epsilon} y$ to mean $x \leq (1 + \epsilon)y$. For any real $x \geq 1$, define $\operatorname{Log} x = \log_2(1 + x)$.

## 2.2 Dominance

A point $p \in \mathbb{R}^d$ *dominates* $q \in \mathbb{R}^d$ (written $p \succ q$) if $p \neq q$ and $p[i] \geq q[i]$ for all $i \in [d]$.

The optimal monotone error is

$$k^* \;=\; \min_{h \in \mathbb{H}_{\text{mon}}} \operatorname{err}_P(h). \tag{1}$$

The expected cost and error of a randomised algorithm $\mathcal{A}$ are

$$\text{expected cost} = \mathbf{E}[X], \tag{2}$$
$$\text{expected error} = \mathbf{E}[\operatorname{err}_P(h)]. \tag{3}$$

# 3 Results

Insert your main results here. Use the theorem/lemma boxes from above.

| Setting | Upper bound | Lower bound |
|---|---|---|
| $\epsilon = 0$ | $O(n)$ | $\Omega(n)$ |
| $\epsilon \in (0, 1)$ | $O(n/\epsilon)$ | $\Omega(n/\epsilon)$ |
| $\epsilon \geq 1$ | $O(\log n)$ | $\Omega(\log n)$ |

Table 1: Summary of upper and lower bounds.

# 4 Algorithms

Describe your algorithms here. Use `\begin{enumerate}` for steps.

**Step 1.** Probe a random element $p \in P$ and observe label$(p)$.

**Step 2.** Update the current best classifier $h$.

**Step 3.** Repeat until $\operatorname{err}_P(h) \leq (1 + \epsilon)k^*$.

# 5 Proofs

Place detailed proofs here. Each proof should reference a labelled theorem or lemma, e.g. *Proof of Theorem 1.1.*

# References

# References

[1] Y. Tao, "Monotone Classification," *PODS'18.*

[2] Y. Tao, "Monotone Classification (Extended)," *PODS'21.*